p0901r0
Size feedback in operator new

Published Proposal, 5 February 2018

This version:
http://wg21.link/P0901R0

Authors:
Andrew Hunter (Google)
Chris Kennelly (Google)

Audience:
EWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Provide access to actual malloc buffer sizes for users.

Table of Contents

§ 1. Motivation

Throughout this document "malloc" refers to the **implementation** *of* `::operator new` *both as fairly standard practice for implementors, and to make clear the distinction between the interface and the implementation.*

Everyone's favorite dynamic data structure, `std::vector`, allocates memory with code that looks something like this (with many details, like `Allocator`, templating for non `char`, and exception safety, elided):

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  const size_t bytes = new_cap;
  void *newp = ::operator new(new_cap);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

Consider the sequence of calls:

```
std::vector<char> v;
v.reserve(37);
// ...
v.reserve(38);
```

All reasonable implementations of malloc round sizes, both for alignment requirements and improved performance. It is extremely unlikely that malloc provided us exactly 37 bytes. We do not need to invoke the allocator here...except that we don't know that for sure, and to use the 38th byte would be undefined behavior. We would like that 38th byte to be usable without a roundtrip through the allocator.

This paper proposes an API making it safe to use that byte, and explores many of the design choices (not all of which are obvious without implementation experience.)

1.1. nallocx: not as awesome as it looks

The simplest way to help here is to provide an informative API answering the question "If I ask for N bytes, how many do I actually get?" [Jemalloc] calls this `nallocx`. We can then use that hint as a smarter parameter for operator new:

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  const size_t bytes = nallocx(new_cap, 0);
  void *newp = ::operator new(bytes);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

This is a good start, and does in fact work to allow vector and friends to use the true extent of returned objects. But there are three significant problems with this approach.

1.1.1. nallocx must give a conservative answer

While many allocators have a deterministic map from requested size to allocated size, it is by no means guaranteed that all do. Presumably they can make a reasonably good guess, but if two calls to `::operator new(37)` might return 64 and 128 bytes, we'd definitely rather know the right answer, not a conservative approximation.

1.1.2. nallocx duplicates work

Allocation is often a crucial limit on performance. Most allocators compute the returned size of an object as part of fulfilling that allocation...but if we make a second call to `nallocx`, we duplicate all that communication, and also the overhead of the function call.

1.1.3. nallocx hides information from malloc

The biggest problem (for the authors) is that `nallocx` discards information malloc finds valuable (the user's intended allocation size.) That is: in our running example, malloc normally knows that the user wants 37 bytes (then 38), but with `nallocx`, we will only ever be told that they want 40 (or 48, or whatever `nallocx(37)` returns.)

Google's malloc implementation (tcmalloc) rounds requests to one of a small (<100) number of *sizeclasses*: we maintain local caches of appropriately sized objects, and cannot do this for every possible size of object. Originally, these sizeclasses were just reasonably evenly spaced among the range they cover. Since then, we have used extensive telemetry on allocator use in the wild to tune these choices. In particular, as we know (approximately) how many objects of any given size are requested, we can solve a fairly simple optimization problem to minimize the total internal fragmentation for any choice of N sizeclasses. The exact results we've achieved are confidential, but the savings are significant.

(sufficiently widespread use of) `nallocx` breaks this. By the time tcmalloc's telemetry sees a request that was hinted by nallocx, to the best of our knowledge the user *wants* exactly as many bytes as we currently provide them. If a huge number of callers wanted 40 bytes but were currently getting 48, we'd lose the ability to know that and optimize for it.

Note that we can't take the same telemetry from `nallocx` calls: we have no idea how many times the resulting hint will be used (we might not allocate at all, or we might cache the result and make a million allocations guided by it.) We would also lose important information in the stack traces we collect from allocation sites.

Optimization guided by malloc telemetry has been one of our most effective tools in improving allocator performance. It is important that we fix this issue *without* losing the ground truth of what a caller of `::operator new` wants.

These three issues explain why we don't believe `nallocx` is a sufficient solution here.

1.2. after allocation is too late

Another obvious suggestion is to add a way to inspect the size of an object returend by `::operator new`. Most mallocs provide a way to do this; jemalloc calls it `sallocx`. Vector would look like:

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  void *newp = ::operator new(new_cap);
  const size_t bytes = sallocx(newp);
  memcpy(newp, ptr_, capacity_);
  ptr_ = newp;
  capacity_ = bytes;
}
```

This is worse than nallocx. It fixes the non-constant size problem, and avoids a feedback loop, but the performance issue is worse (this is the major issue *fixed* by [SizedDelete]!), and what's worse, the above code invokes UB as soon as we touch byte `new_cap+1`. We could in principle change the standard, but this would be an implementation nightmare.

§ 1.3. realloc's day has passed

We should also quickly examine why the classic C API `realloc` is insufficient.

```
void vector::reserve(size_t new_cap) {
  if (capacity_ >= new_cap) return;
  ptr_ = realloc(ptr_, new_cap);
  capacity_ = new_cap;
}
```

In principle a realloc from 37 to 38 bytes wouldn't carry the full cost of allocation. But it's dramatically more expensive than making no call at all. What's more, there are a number of more complicated dynamic data structures that store variable-sized chunks of data but are never actually resized. These data structures still deserve the right to use all the memory they're paying for.

Furthermore, `realloc`'s original purpose was not to allow the use of more bytes the caller already had, but to (hopefully) extend an allocation in place to adjacent free space. In a classic malloc implementation this would actually be possible...but most modern allocators use variants of slab allocation. Even if the 65th byte in a 64-byte allocation isn't in use, they cannot be combined into a single object; it's almost certainly required to be used for the next 64-byte allocation. In the modern world, `realloc` serves little purpose.

§ 2. Proposal

We propose adding new overloads of `::operator new` that directly inform the user of the size available to them.

```
struct std::return_size_t {};

struct std::sized_ptr_t {
  void *p;
  size_t n;
};

std::sized_ptr_t ::operator new(size_t size, const std::return_size_t&);
std::sized_ptr_t ::operator new(size_t size, std::align_val_t al, const std
std::sized_ptr_t ::operator new(size_t size, const std::nothrow_t &, const
std::sized_ptr_t ::operator new(size_t size, std::align_val_t al, const std
```

(We would need the same `::operator new[]` overloads; omitted for clarity.) Another signature we could use would be:

```
enum class return_size_t : std::size_t {};
void * ::operator new(size_t size, std::return_size_t&);
```

(and so on.) This is slightly simpler to read as a signature, but arguably worse in usage:

```
std::tie(obj.ptr, obj.size) = ::operator new(37, std::return_size_t{});

// ...vs...

// Presumably the object implementation wants to contain a size_t,
// not a return_size_t.
std::return_size_t rs;
obj.ptr = ::operator new(37, rs);
obj.size = rs;
```

More importantly, this form is less efficient. In practice, underlying malloc implementations provide actual definitions of `::operator new` symbols which are called like any other function. Passing a reference parameter requires us to actually return the size via memory; most important ABIs support

returning at least two scalar values in registers (even if they're members of a trivially copyable struct) which can be dramatically more efficient.

Whether we use a reference parameter or a second returned value, the interpretation is the same. Candidate (rough) language for the first overload would be:

```
[[nodiscard]] std::sized_ptr_t ::operator new(size_t size, const
std::return_size_t&);
```

> *Effects*: returns a pair (p, n) with `n >= size`. Behaves as if `p` was the return value of a call to `::operator new(n)`.

The intention is quite simple: we return the "actual" size of the allocation, and rely on "as if" to do the heavy lifting that lets us use more than `size` bytes of the resulting allocation. **In particular, this means at no point do we risk undefined behavior from using more bytes than `::operator new` was called with.**

§ 2.1. *How* many operator news?

It is unfortunate that we have so many permutations of `::operator new`--eight seems like far more than we should really need! But there really isn't any siginificant runtime cost for having them.

Any alternate proposal that didn't require so many trivially different signatures would be appreciated.

§ 2.2. Implementation difficulty

It's worth reiterating that there's a perfectly good trivial implementation of these functions:

```
std::sized_ptr_t ::operator new(size_t n, const std::return_size_t&) {
  return {::operator new(n), n};
}
```

Malloc implementations are free to properly override this with a more impactful definition, but this paper poses no significant difficulty for toolchain implementors.

§ 2.3. Advantages

It's easy to see that this approach nicely solves the problems with `nallocx` or the like. We pay almost nothing in speed to return an actual-size parameter; allocator telemetry knows actual request sizes exactly; and we are told exactly the size we have, without risk of UB.

§ 3. Related work

[AllocatorExt] considered this problem at the level of the `Allocator` concept. Ironically, the lack of the above API was one significant problem: how could an implementation of `std::allocator` provide the requested feedback in a way that would work with any underlying malloc implementation?

If this proposal is accepted, it's likely that [AllocatorExt] should be taken up again.

§ References

§ Informative References

[AllocatorExt]
Jonathan Wakely. Extensions to the Allocator interface. 2015-07-08. URL: http://wg21.link/P0401R0

[Jemalloc]
jemalloc(3) - Linux man page. URL: http://jemalloc.net/jemalloc.3.html

[SizedDelete]
L; et al. C++ Sized Deallocation. URL: http://wg21.link/n3536